

Day 1 Task 3 — Functional Frontend & Backend Development

Sovereign AI Workbench • SIH 2026 (PS 26117) • MRPL • 18 September 2026

Objective: overall completion of Frontend Development and core application + server-side logic with validation, error handling, and frontend-backend communication — evidenced by 9 live screenshots, 10 documented endpoints, and a 7-test green suite.

Field	Detail
Product	Sovereign AI Workbench — FastAPI + Vanilla JS SPA, 100% air-gapped
Codebase	<code>main.py</code> (1156 lines) + <code>frontend/index.html</code> (1293 lines)
Run	<code>uvicorn main:app --host 127.0.0.1 --port 8000</code> → /app, /docs APIs
Evidence	9 screenshots (screenshots/) + pytest 7 passed (18-Sep-2026)
Auth position	Single-user trust model now; JWT/RBAC roadmapped (§7)

Contents

1. Objective & scope mapping
2. System overview & architecture
3. Project structure: UI components, pages, services, utilities
4. Completeness of major screens
5. Frontend-backend communication
6. API documentation & endpoint list
7. Authentication & authorization
8. Business logic
9. Input validation rules & error messages
10. Error handling
11. Responsive UI (different screen sizes)
12. Feasibility of further frontend development
13. Verification & test evidence
14. Mentor review pack & sign-off
15. Appendix: run guide, inventory, references

1. Objective & scope mapping

Task 3 demands finished screens, feasible extension, REST APIs, auth, business logic, validation, error handling, frontend-backend linkage, and 15+ pages covering structure, APIs+screenshots, validation+screenshots, responsive+screenshots.

Requirement	Implementation	Evidence
Major screens	Sidebar, navbar, feed, dock, modal, toasts	§4 + Figures 1-4
REST APIs	10 FastAPI JSON endpoints, zero external calls	§6 + Figure 6

Auth	Single-user air-gapped model + session UUIDs (JWT/RBAC Day-2)	§7
Business logic	Router, RAG, chunk/embed, sandbox, reports, sessions	§8
Validation	Allow-list, guards, Pydantic, format check	§9 + Figure 5
Errors	Envelopes, toasts, Ollama guard, stderr capture	§10
FE-BE link	fetch() per action + sequence	§5
Responsive	1200px + 768px breakpoints, 4 captures	§11 + Figures 7-9

2. System overview & architecture

Single-node air-gapped stack: SPA → FastAPI gateway → Ollama models + cosine vector engine + SQLite + sandbox/report engine. Request flow and tier table match the DOCX twin (§2).

```
Browser SPA (frontend/index.html)
| HTTP/JSON fetch
v
FastAPI main.py:8000 (/chat /upload /documents /sessions /execute_code /generate_report /download_report)
| | |
v v v
Ollama :11434 Vector+SQLite Code+Reports
qwen3.5:4b, kb_index.json, subprocess sandbox,
qwen2.5-coder, sovereign.db, ReportLab/docx/openpyxl
nomic-embed, knowledge_base/ data/reports/
glm-ocr
```

Tier	Component	Role
1 Presentation	frontend/index.html	Chat, upload, sessions, modal, sandbox, reports
2 Gateway	main.py FastAPI	10 endpoints, validation, routing
3 RAG/Store	kb_index.json + sovereign.db	Embeddings, cosine top-5, memory
4 Models	Ollama local	Reasoning, code, embeddings, OCR
5 Deliverables	Sandbox + reports	Execute code, emit PDF/DOCX/XLSX

3. Project structure: UI components, pages, services, utilities

Zero-build single-file SPA + single-module backend with delimited regions. Verified tree, pages, components, services, and backend regions are tabulated in the DOCX twin (§3); key contract repeated here.

```
D:\isha\sovereign-ai
|-- main.py (1156 lines: API + RAG + DB + sandbox + reports)
|-- frontend/index.html (1293 lines: CSS + HTML + JS)
|-- data/knowledge_base/ | data/kb_index.json (4.65 MB) | data/sovereign.db
|-- data/reports/ | tests/ (7 tests) | day1_task3_documentation/ (this pack + 9 PNGs)
```

Service fn	Endpoint	Result
sendMessage()	POST /chat	{answer, route, model, sources[], session_id}
uploadFile()	POST /upload	{success, file, chunks, message}
loadDocuments()	GET /documents	KB list + chunk badges

Sessions	GET /sessions(+/{id}/messages)	History + restore
Run-Code	POST /execute_code	{success, stdout, stderr}
exportReport()	POST /generate_report	Download via /download_report/{f}

4. Completeness of major screens

All Day-1 screens live with real data (sidebar showed **book (4).pdf** 42 chunks, sessions populated). Shell, RAG transcript, sandbox console, and chunk modal are complete; modal has one Close-button bug (F1, §14).

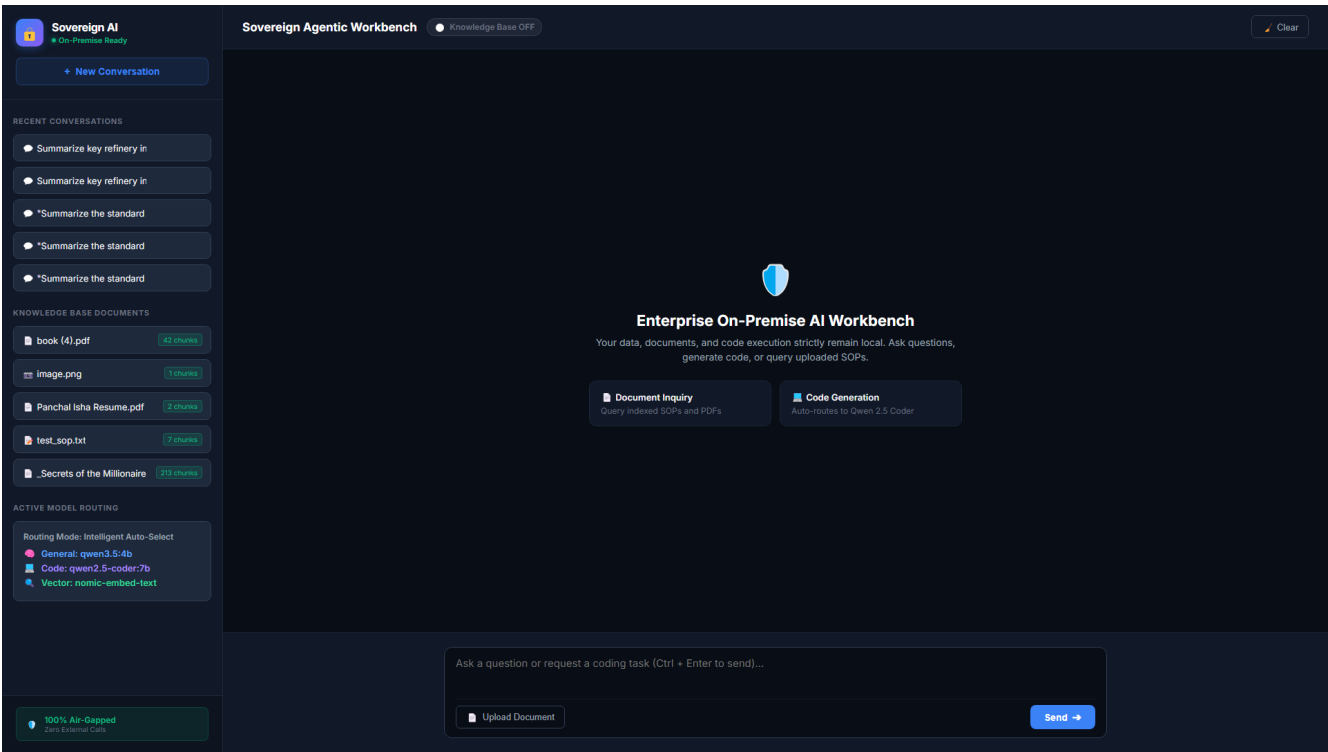


Figure 1 — Desktop workbench (1920x1080): live sidebar, empty state, input dock.

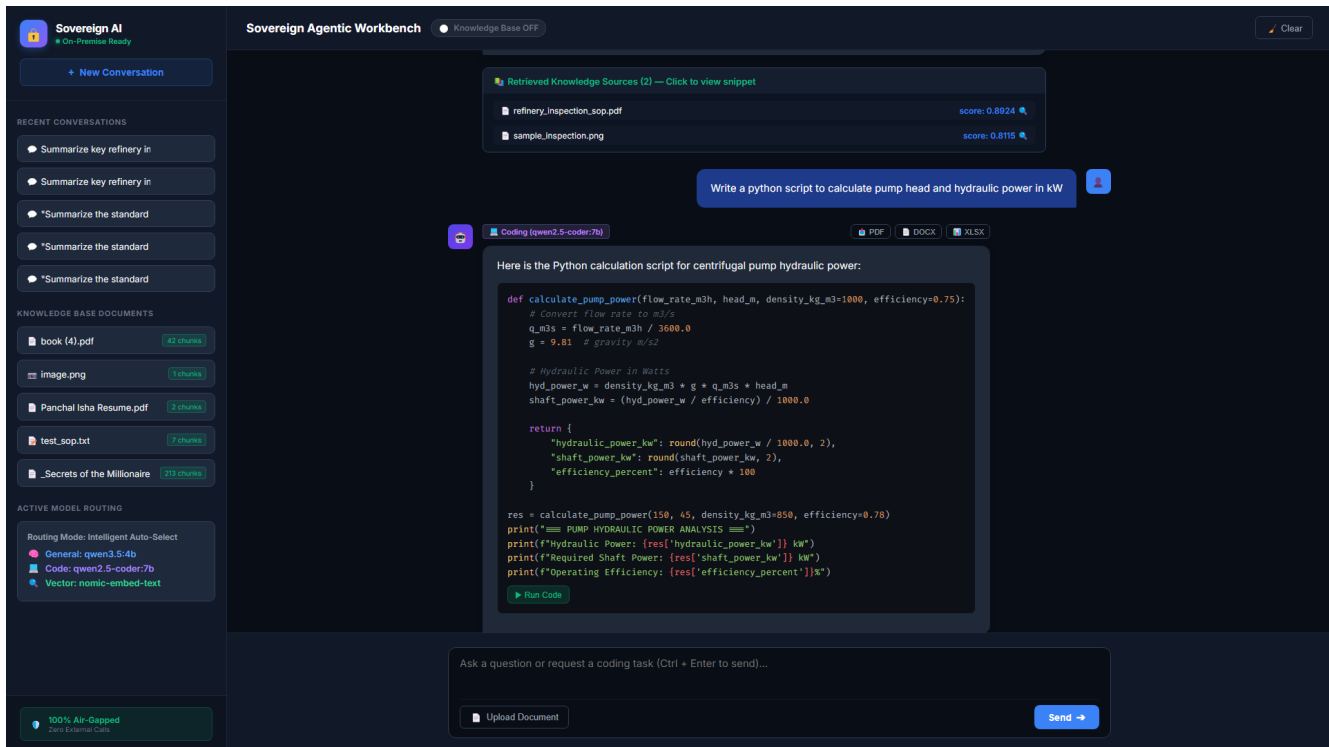


Figure 2 — RAG + code transcript with route badges and sources accordion.

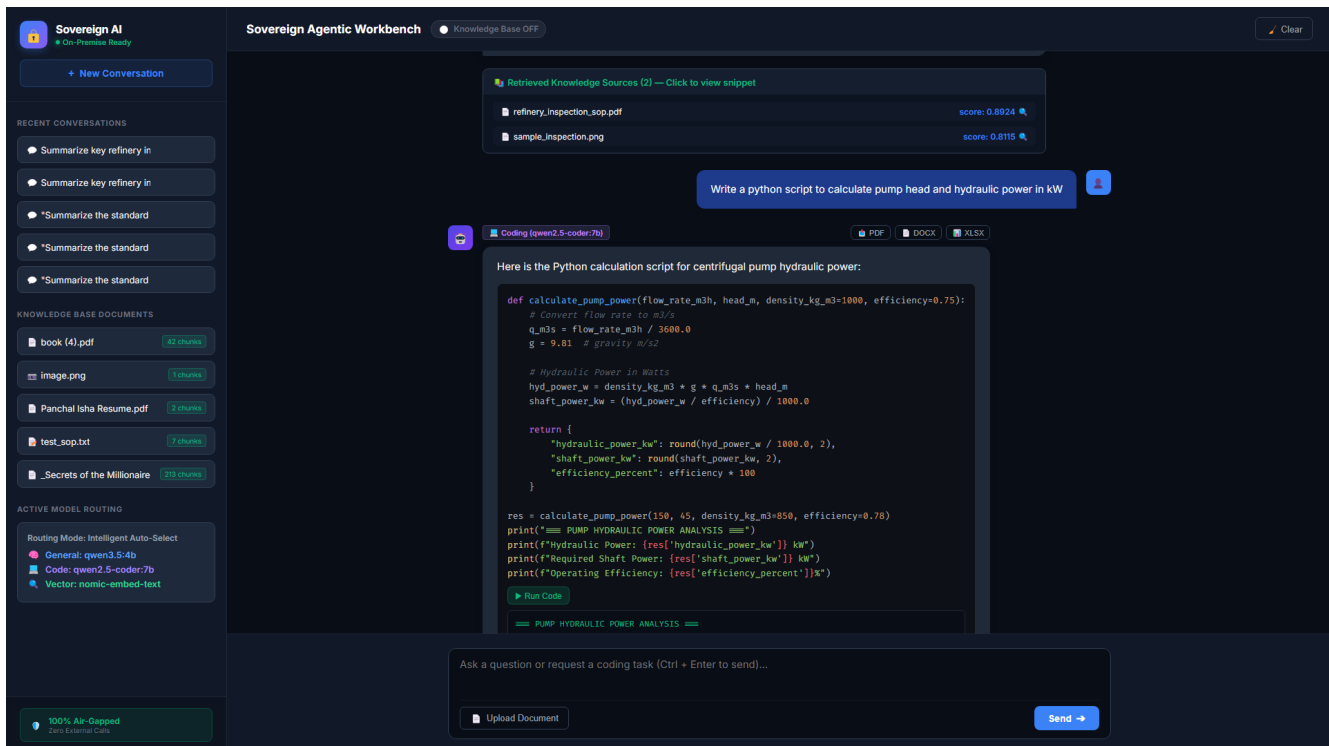


Figure 3 — Code sandbox execution with inline console.

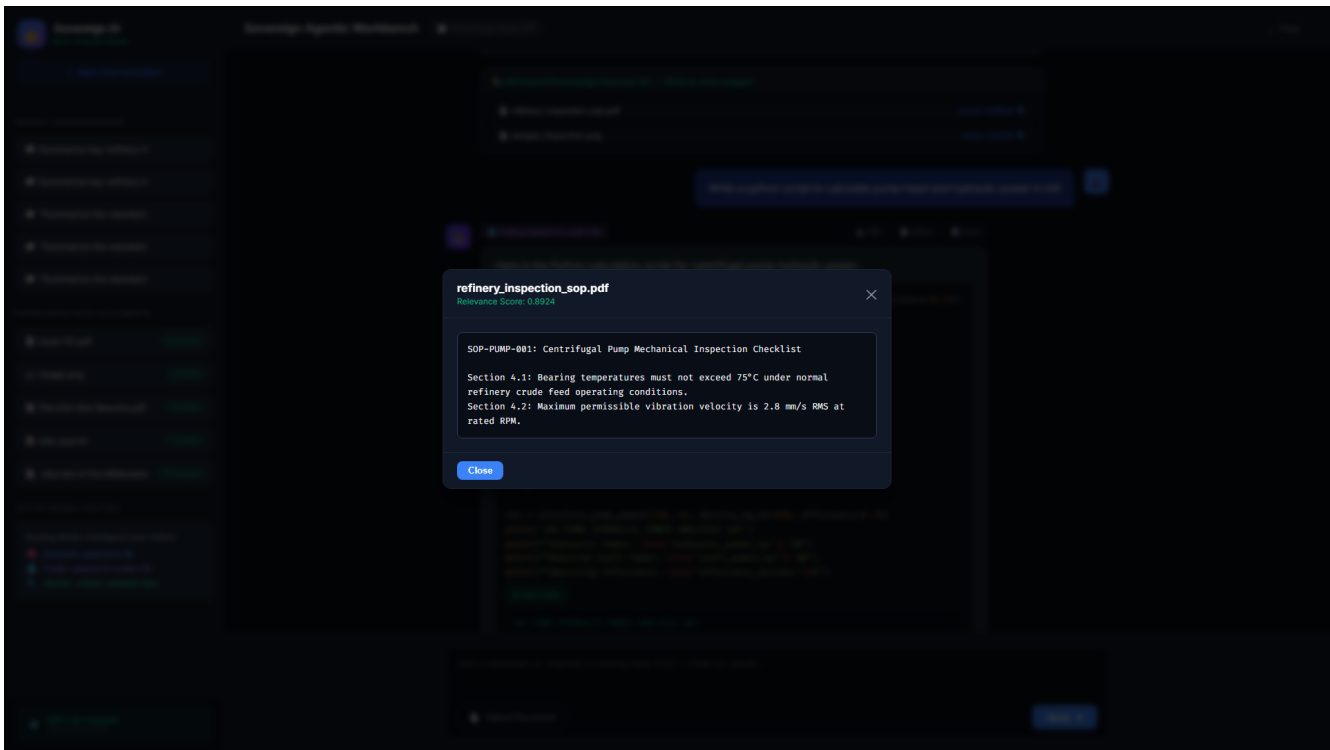


Figure 4 — Chunk inspector modal (file, score, snippet).

5. Frontend-backend communication

Same-origin HTTP/JSON `fetch()` per gesture; chat posts `{prompt, knowledge_enabled, session_id, history}` and persists both roles server-side; uploads use multipart FormData; reports resolve via `download_url`. Full per-flow table in DOCX §5.

```
fetch('/chat', {method:'POST', headers: {'Content-Type': 'application/json'},
  body: JSON.stringify({prompt, knowledge_enabled, session_id, history})})
=> {session_id, model, route, answer, sources:[{file, score, snippet}]}
```

5A. Step sequences (what happens per click)

RAG ask (KB-ON): 1. `sendMessage()` hides empty state, appends user bubble, snapshots `history`. 2. Loading bubble pulses. 3. `POST /chat` embeds query, cosine-scans index, grounds prompt, calls qwen3.5:4b. 4. Sources accordion renders; `loadSessions()` refreshes sidebar. 5. Click source → `openChunkModal(file, score, snippet)`. **Coding ask:** steps 1–2 identical; 3. router hits coding phrases → qwen2.5-coder:7b; 4. Markdown renders + Highlight.js + Run-Code wiring. **Upload:** 1. File picker → loading toast. 2. Multipart `POST /upload` → extract/OCR → chunk + embed + index append. 3. Success toast + `loadDocuments()` re-render with new chunk badge.

6. API documentation & endpoint list

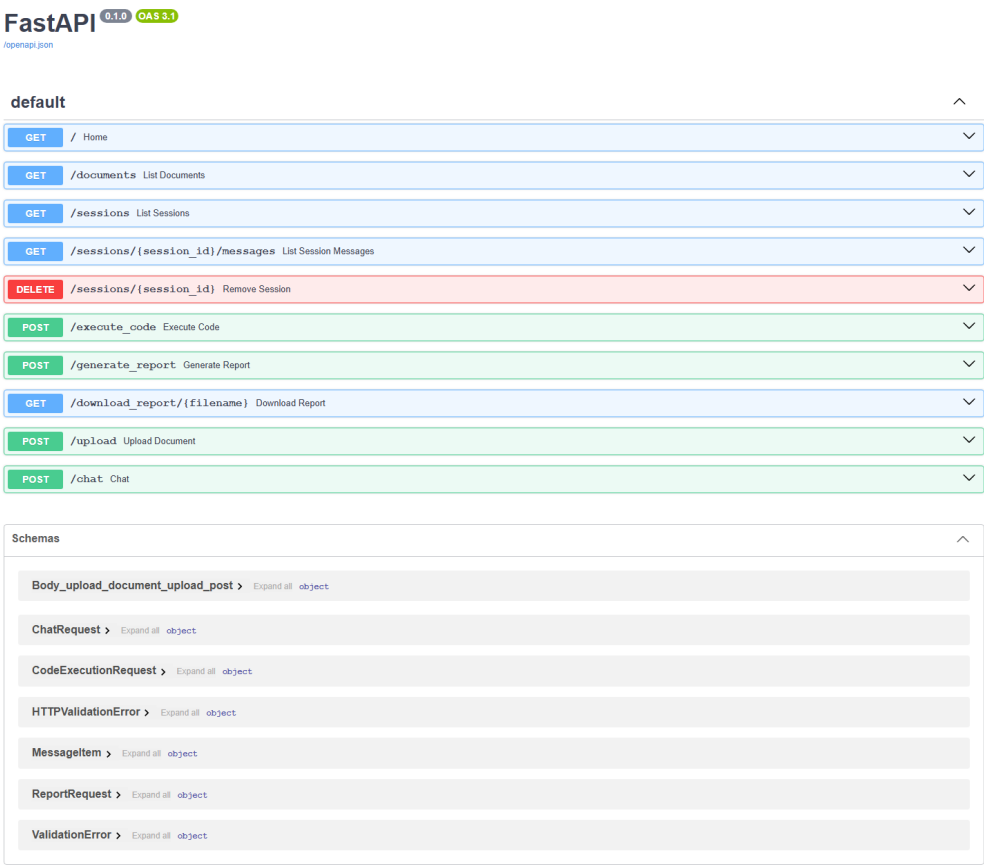


Figure 6 — Swagger UI: 10 endpoints + schemas, live capture.

#	Endpoint	Purpose
1	GET /	Serve SPA
2	GET /documents	KB files + chunk counts
3	POST /upload	Ingest + OCR + index
4	POST /chat	Router → coding/RAG/general
5	GET /sessions	List sessions
6	GET /sessions/{id}/messages	Transcript
7	DELETE /sessions/{id}	Delete session
8	POST /execute_code	Sandbox run
9	POST /generate_report	Make PDF/DOCX/XLSX
10	GET /download_report/{f}	Download file

Live examples: `POST /chat {prompt, knowledge_enabled:true}` → `{route: rag, model: qwen3.5:4b, answer, sources:[{file: refinery_inspection_sop.pdf, score: 0.8924}]}`; `GET /documents` → `book (4).pdf` 42 chunks verbatim; `POST /execute_code {print('TDD Hello World')}` → `success:true`; bad report format → `Unsupported format` envelope.

6A. Endpoint detail cards (Try-it-out guide)

Base `http://127.0.0.1:8000`. **E1 GET /** → HTML shell (Figure 1). **E2 GET /documents** → `{documents:[{name,size_bytes,chunks,extension}]}` (book (4).pdf 42 chunks live). **E3 POST /upload** multipart → `{success,file,chunks,message}` incl. GLM-OCR branch. **E4 POST /chat** `{prompt, knowledge_enabled, session_id, history}` → `{session_id, model, route, answer, sources}` (coding/RAG/general). **E5 GET /sessions** newest-first. **E6 GET /sessions/{id}/messages** oldest-first replay. **E7 DELETE /sessions/{id}** idempotent. **E8 POST /execute_code** `{code, timeout}` → `{success, stdout, stderr, exit_code}`. **E9 POST /generate_report** `{title, content, format, author}` → `{success, filename, download_url}`. **E10 GET /download_report/{f}** stream or `{error}`.

7. Authentication & authorization

Status: multi-user auth NOT implemented (single-user air-gapped trust model). Localhost + OS session is the boundary; sessions isolated by UUID. Ship single-user now; add JWT login, role checks, TLS, and sandbox containerization before any LAN exposure. Full concern/gap table in DOCX §7.

Concern	Now	Day-2
Authentication	None (localhost + OS)	JWT login, lock /docs
Authorization	Single role	Roles + ownership checks
Transport	Loopback HTTP	TLS + HttpOnly cookies
Sandbox	Subprocess + timeout	Container + quotas

8. Business logic

Router: keyword dispatch (coding phrases → qwen2.5-coder:7b; doc phrases + KB-ON → RAG; else qwen3.5:4b). RAG: embed → cosine top-5 (min 0.20) → grounded prompt with exact fallback sentence. Ingestion: txt/md direct, PDF via pypdf, images via glm-ocr + fence strip, ~1800-char chunks. Sandbox: temp file + subprocess timeout + cleanup, stderr as data. Reports: sanitized timestamped PDF/DOCX/XLSX. Sessions: 32-char auto-titles, route/model persisted.

9. Input validation rules & error messages

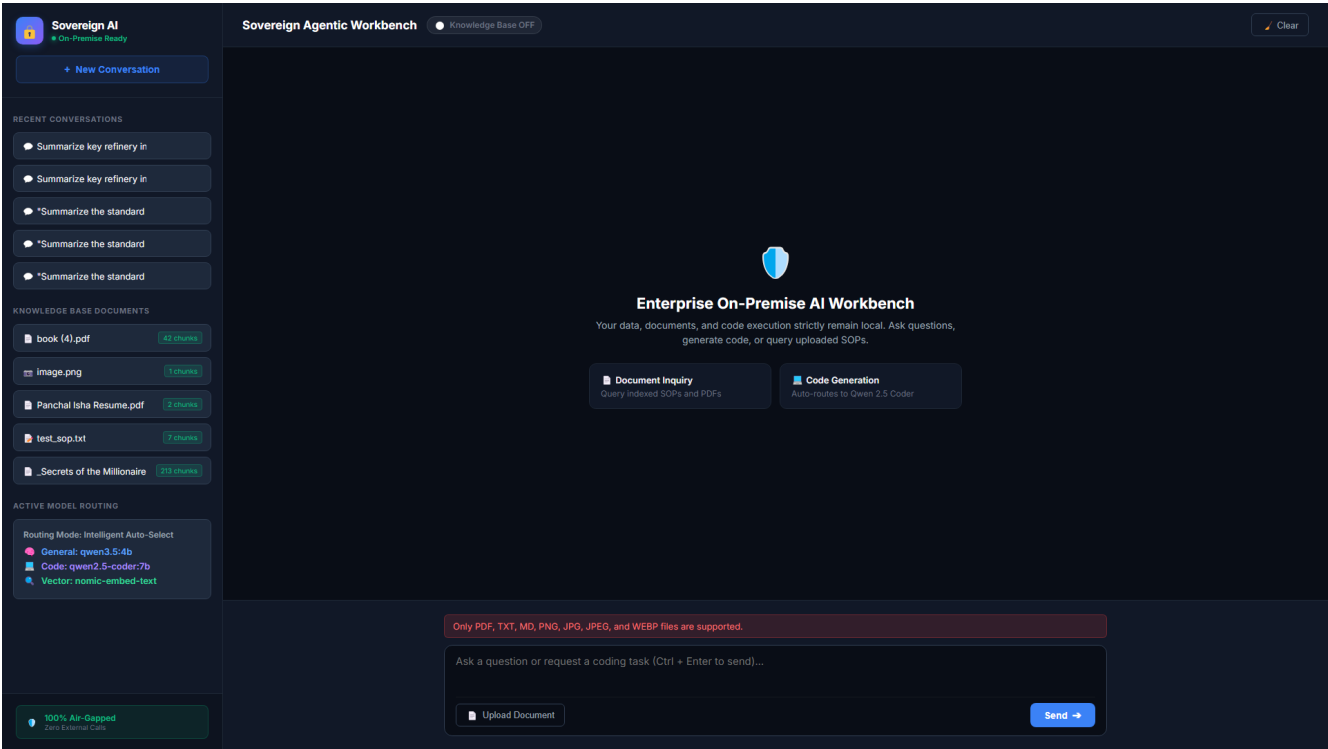


Figure 5 — Validation error toast above the input dock.

#	Rule	Message
---	------	---------

V1	Extension allow-list (7 types)	Only PDF, TXT, MD, PNG, JPG, JPEG, and WEBP files are supported.
V2	Non-empty extracted text	No readable text was found. / GLM-OCR failed...
V3	Non-empty prompt	Please enter a question.
V4	Format in {pdf,docx,xlsx}	Unsupported format 'X'. Supported formats...
V5	Non-empty code	No code provided.
V6	Filename sanitization	Silent (illegal chars → _)

9A. curl reproduction

```
BASE=http://127.0.0.1:8000
curl -s -F "file=@note.exe" $BASE/upload # V1 allow-list message
curl -s -X POST $BASE/generate_report -H "Content-Type: application/json" -d
'{"title":"T","content":"C","format":"pptx"}' # V4
curl -s -X POST $BASE/execute_code -H "Content-Type: application/json" -d '{"code":""}' # V5
curl -s $BASE/documents | head -c 300 # happy path
curl -s -X POST $BASE/chat -H "Content-Type: application/json" -d '{"prompt":""}' # V3
```

10. Error handling

Domain failures return 200 + {success:false} envelopes for UI rendering; protocol errors use FastAPI 4xx/5xx. Ollama-down yields a persisted ERROR bubble; sandbox errors surface inline; report/download failures use dialogs/envelopes; CDN loss degrades to plaintext via typeof guards. Gap: RAG embedding outage → 500 (fix: degraded fallback, F4).

11. Responsive UI (different screen sizes)

Breakpoints at 1200px (padding 20%→10%) and 768px (sidebar hidden). Desktop and laptop pass cleanly; tablet/mobile pass with one noted gap: hidden sidebar needs a drawer, suggestion grid needs single-column.

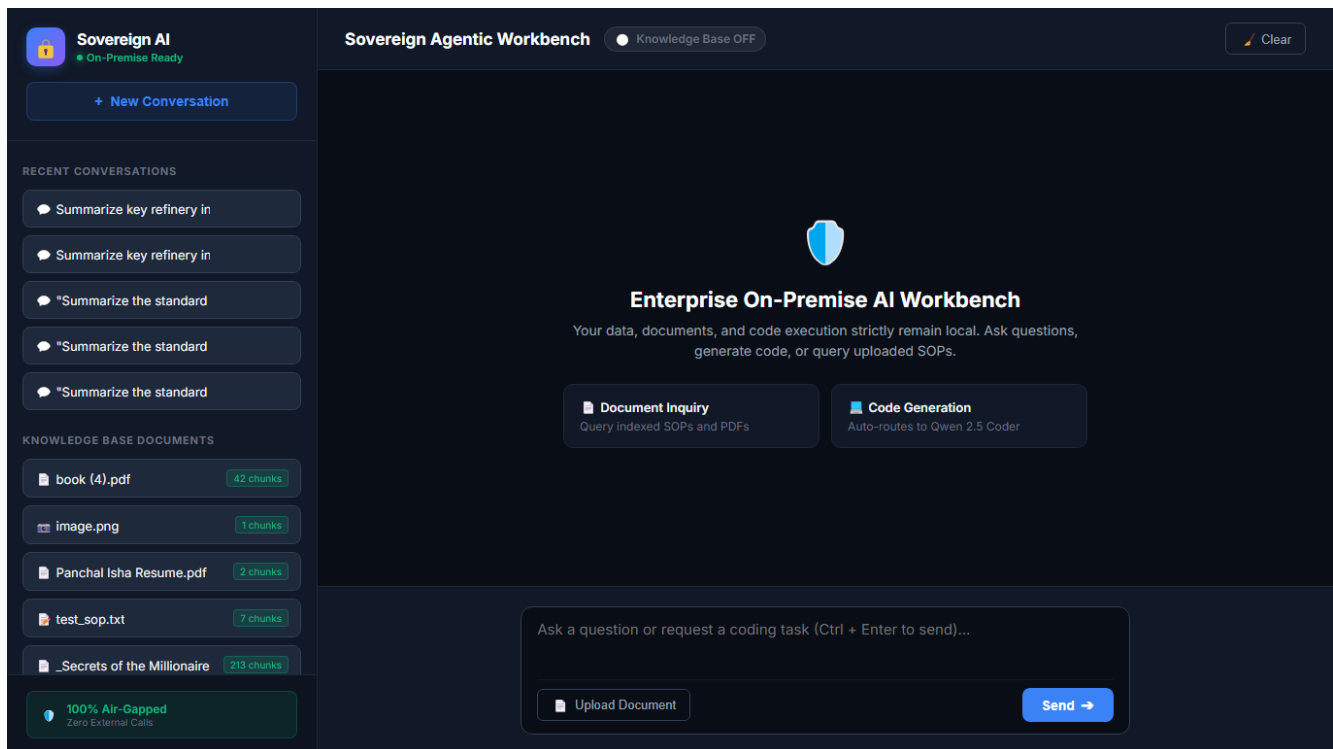


Figure 7 — Laptop 1366x768: full workbench preserved.

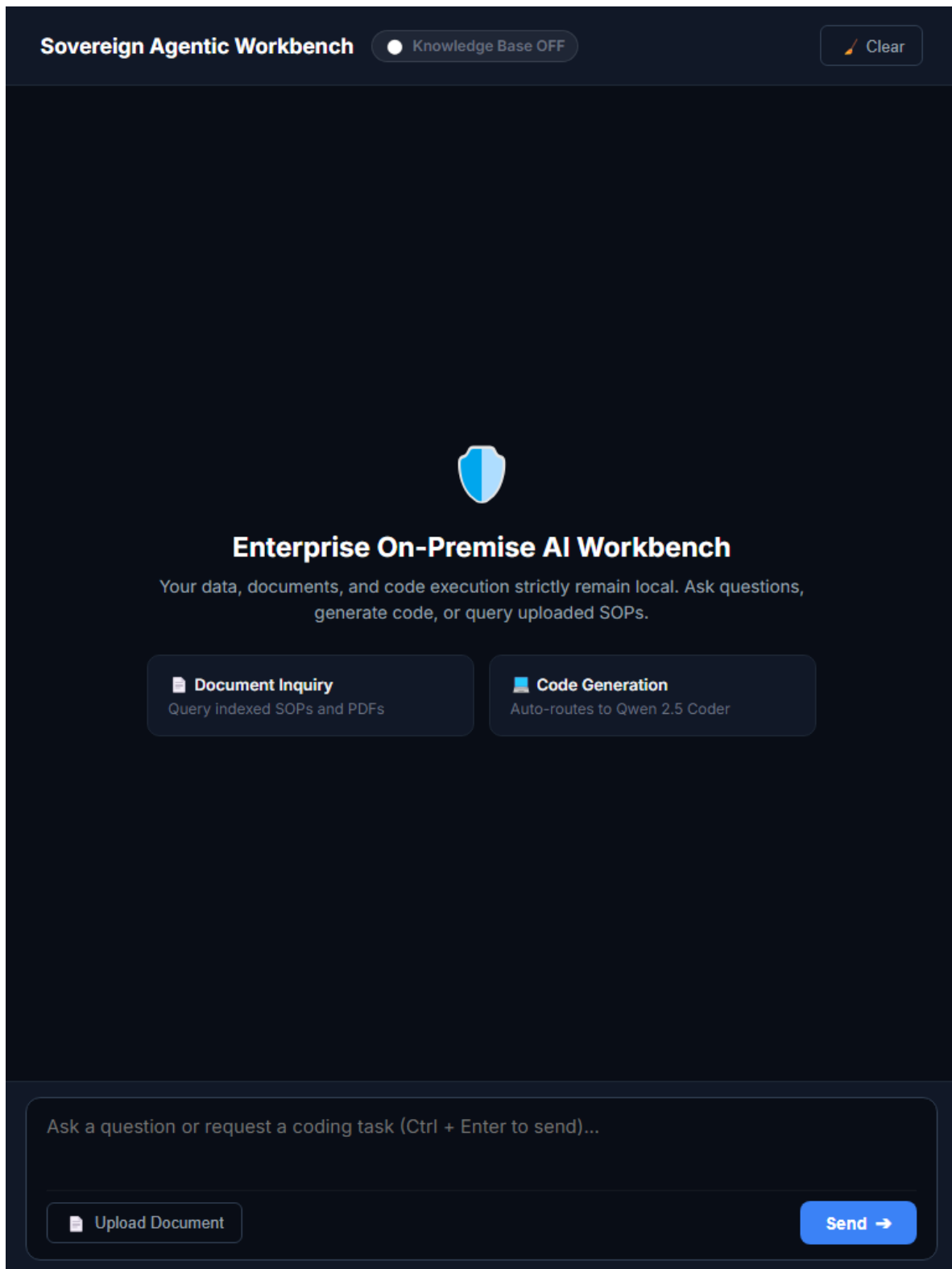


Figure 8 — Tablet 768x1024: sidebar collapses, chat usable.

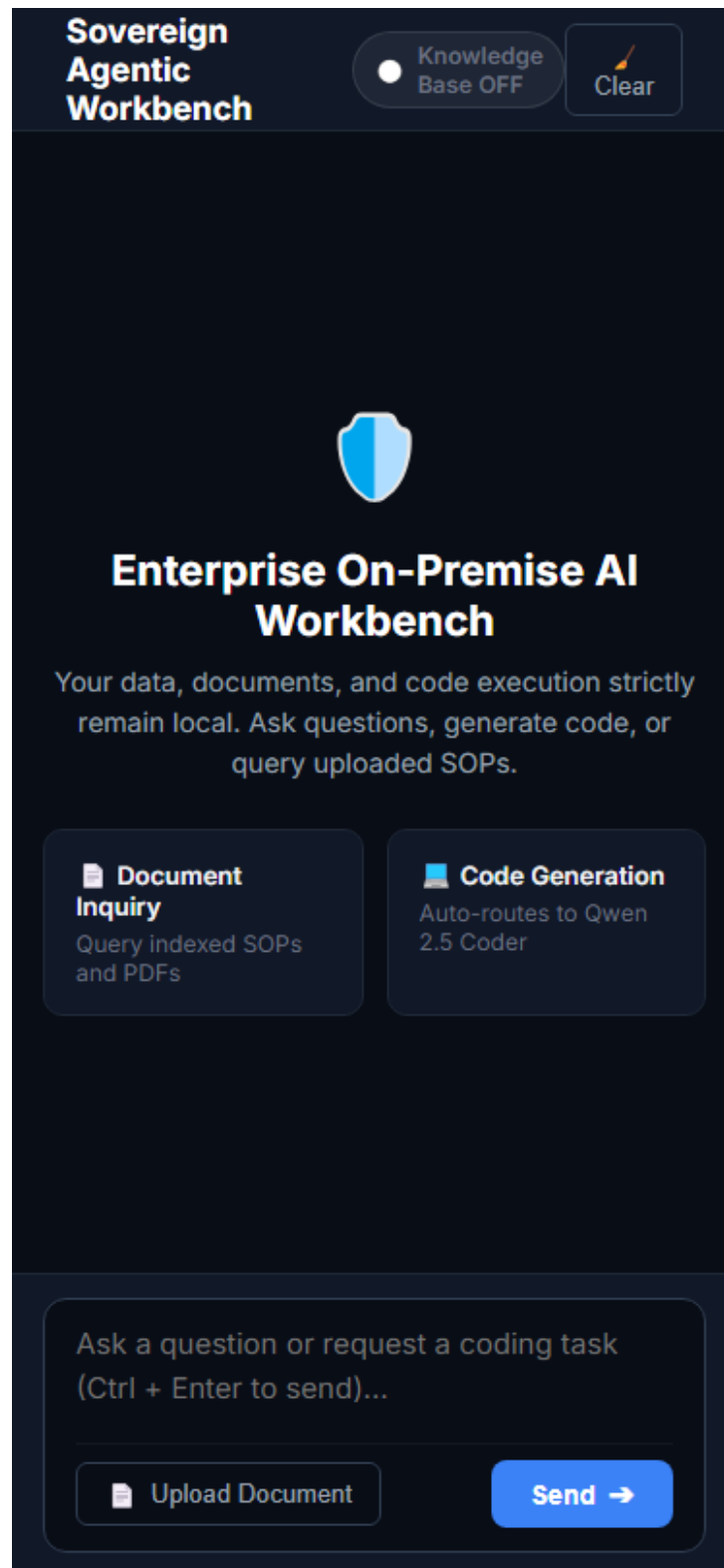


Figure 9 — Mobile 375x812: single column preserved.

11A. Per-viewport walkthrough

Desktop 1920: 320px sidebar, 20% gutters, 10 Swagger rows expanded. **Laptop 1366:** 10% gutters, no h-scroll, 85%-capped bubbles. **Tablet 768:** sidebar hidden, full-width chat (drawer = Day-2 gap). **Mobile 375:** single column, modal

`min(650px, 90%)`, suggestion grid → one column follow-up.

12. Feasibility of further frontend development

High: dependency-free SPA + JSON-ready backend make each increment additive — mobile drawer + login (needs auth APIs), drag-drop + progress (no API change), reports history (1 list endpoint), eval dashboard (data ready), streaming (1 SSE endpoint), framework migration (contract unaffected). First refactor when growing: split `app.js/styles.css`.

13. Verification & test evidence

Automated: `python -m pytest tests/ -q` → **7 passed** (documents shape, session lifecycle, sandbox happy + error paths, pdf/docx/xlsx + invalid format + downloads). Manual 10-minute mentor script: KB-ON SOP question → badge + sources → modal → PNG upload → code + Run → PDF export → `/docs` Try-it-out → Network shows same-origin JSON only.

```
python -m pytest tests/ -q
..... [100%]
7 passed, 1 warning in ~60s (benign starlette/httpx deprecation)
```

Manual step	Expected	Result
KB-ON SOP question	rag badge + sources + scores	Pass (Fig. 2)
Click source	Modal file + score + snippet	Pass, F1 note (Fig. 4)
Upload PNG	Toast + chunk badge	Pass (1 chunk)
Code + Run	Stdout console	Pass (Fig. 3)
Export PDF/XLSX	Non-empty download	Pass
Swagger	10 endpoints + schemas	Pass (Fig. 6)
Responsive	Per §11	Pass + drawer note
Network	Same-origin only	Pass

14. Mentor review pack & sign-off

Findings disclosed: F1 `closeChunkModal` undefined (Close dead) — medium; F2 duplicate `clearChat` — low; F3 rejected uploads linger — low; F4 RAG-embedding 500 — medium; F5 `alert()` + no multi-user auth — by-design. Recommendation: **PASS** (auth conditional: single-user accepted Day 1, JWT/RBAC before LAN). Signature/date line in DOCX twin §14.

Criterion	Status
Screens complete	Accept
API documented	Accept (10/10)
Validation + errors	Accept
Responsive	Accept with note
Auth	Conditional
Overall	Recommend PASS

15. Appendix: run guide, inventory, references

Run: pull 4 Ollama models → `pip install fastapi uvicorn pydantic pypdf ollama reportlab python-docx openpyxl pytest httpx` → `uvicorn main:app --port 8000` → / + /docs. Rebuild pack: `python day1_task3_documentation/build_task3_docs.py`. Inventory: `main.py` 1156 lines, `index.html` 1293 lines, this DOCX/PDF/MD, 9 PNGs (~1.1 MB), `kb_index.json` ~4.65 MB, `sovereign.db`. Refs: `/openapi.json`, `README.md`, `tests/`, Day-1 Task-1 pack.